ENTERPRISE ENGINEERING DOCUMENTATION

Meeting Room Management & Live Meeting Display System

Complete A-to-Z Knowledge Transfer (KT) Technical Manual

Project:	Meeting Room Management & Display
Version:	v1.0 Production Architecture
Framework:	Next.js 16 (App Router), React 19, TypeScript
Database:	PostgreSQL (via Prisma 6.4 ORM)
Authentication:	JWT Session Cookies + Bcrypt Hashing
Primary Admin:	`Pradhan` (Password: `123`)
Date:	August 2026

This technical manual provides complete knowledge transfer for developers, system administrators, and operations teams to maintain, extend, and deploy the Meeting Room System on PostgreSQL and Cloud Hosting.

1. Executive Summary & Architecture Overview

The **Meeting Room Management & Live Meeting Display System** is an enterprise software solution built to schedule conference meetings, assign office rooms, eliminate double-booking conflicts, and project real-time meeting statuses onto lobby TV screens and room-specific outside-the-door tablets.

High-Level System Architecture

Key Technology Stack

Layer	Technology	Purpose & Implementation Details
Frontend	Next.js 16 + React 19 + TypeScript	App Router with server-side rendering and interactive client components.
Styling	Tailwind CSS 4.0 + Lucide React	Neon glassmorphism, responsive data tables, glowing ambient TV display.
Database & ORM	Prisma ORM 6.4 + PostgreSQL	Enterprise relational database with pooling, ACID transactions, and cascades.
Security	BcryptJS + JSON Web Tokens (JWT)	Bcrypt password hashing (10 salt rounds) and secure HTTP-only cookies.
Real-Time Sync	Polling Engine + Time Simulator	15-second dynamic polling and optional simulated time testing toolbar.

2. Database Schema & Data Models

The database schema is defined in `prisma/schema.prisma` using a PostgreSQL datasource. The relations enforce foreign key integrity and prevent orphan records.

Entity Relationship Summary

- **User (1) to Meetings (N):** An employee or admin can organize multiple meetings. If a user is deleted, their meetings retain history via `onDelete: SetNull`.
- **Room (1) to Meetings (N):** A room can host many meetings. When deleting a room with existing meetings, the business logic soft-deactivates the room (`status: inactive`) to preserve historical logs.

Data Dictionary

1. User Model (`users`)

Column Name	Data Type	Constraints	Description
`id`	String (UUID)	PRIMARY KEY	Unique identifier generated automatically.
`userId`	String	UNIQUE, NOT NULL	Human-readable employee ID (e.g. `PRADHAN` , `USR001`).
`name`	String	NOT NULL	Display name (e.g. `Pradhan` , `Rahul Sharma`).
`email`	String	UNIQUE, NOT NULL	Email address (case-insensitive login identifier).
`passwordHash`	String	NOT NULL	Bcrypt encrypted password hash.
`role`	String	DEFAULT('user')	Access role: `"admin"` or `"user"` .
`status`	String	DEFAULT('active')	Account state: `"active"` or `"inactive"` .
`createdAt`	DateTime	DEFAULT(now())	Record creation timestamp.
`updatedAt`	DateTime	AUTO-UPDATE	Last update timestamp.

2. Room Model (`rooms`)

Column Name	Data Type	Constraints	Description
`id`	String (UUID)	PRIMARY KEY	Unique identifier.
`roomName`	String	NOT NULL	Descriptive name (e.g. `Conference Room A`).
`roomNumber`	String	UNIQUE, NOT NULL	Door identifier (e.g. `Room 101` , `R101`).
`location`	String	NOT NULL	Building location (e.g. `1st Floor` , `2nd Floor`).
`capacity`	Integer	NOT NULL	Maximum seating capacity (e.g. `20`).
`status`	String	DEFAULT('active')	State: `"active"` or `"inactive"` .

3. Meeting Model (`meetings`)

Column Name	Data Type	Constraints	Description
`id`	String (UUID)	PRIMARY KEY	Unique identifier.
`title`	String	NOT NULL	Title/Topic of discussion (e.g. `Client Discussion`).
`description`	String	NULLABLE	Optional agenda notes and instructions.
`organizer`	String	NOT NULL	Host display name.
`organizerId`	String	FOREIGN KEY (User)	Reference to User table.
`roomId`	String	FOREIGN KEY (Room)	Assigned room reference.
`meetingDate`	String	INDEXED	Date in `YYYY-MM-DD` format.
`startTime`	String	NOT NULL	24-hour time `HH:mm` (e.g. `14:00`).
`endTime`	String	NOT NULL	24-hour time `HH:mm` (e.g. `15:00`).

3. Business Logic & Core Algorithms

Algorithm 1: Room Conflict & Double-Booking Prevention

Located in `src/lib/meetingStatus.ts`: The system strictly validates time overlap prior to inserting or updating any meeting in the database.

> **Mathematical Overlap Principle:**
> Two time intervals `[S1, E1)` and `[S2, E2)` overlap if and only if:
> `max(S1, S2) < min(E1, E2)`
> Equivalent to SQL query: `(existing.startTime < new.endTime) AND (existing.endTime > new.startTime)`

Step-by-Step Validation Sequence:

1. **Order Check:** Verify that `endTime > startTime`. If `endTime ≤ startTime`, immediately reject with `400 Bad Request`.
2. **Room Active Check:** Verify that the selected room exists and has `status === "active"`. If inactive, reject booking.
3. **Overlap Check:** Query the database for meetings in the same room on the same date:

```
const existingMeetings = await prisma.meeting.findMany({
  where: {
    roomId,
    meetingDate,
    ...(excludeMeetingId ? { id: { not: excludeMeetingId } } : {}),
  },
});

const conflicting = existingMeetings.find(
  (m) => m.startTime < endTime && m.endTime > startTime
);
```

4. **Boundary Rule (Allowed):** If Meeting A is `14:00 - 15:00` and Meeting B is `15:00 - 16:00`, they touch at `15:00` but do not overlap. This is allowed!
5. **Conflict Response:** If conflicting, return `409 Conflict` with a helpful message identifying the conflicting meeting title and time.

Algorithm 2: Dynamic Real-Time Status Calculation

Meeting status is **never manually typed**. It is evaluated dynamically on-the-fly comparing the current time `T` (or simulated time) against the meeting start `S` and end `E`:

Condition	Computed Status	Visual Badge	Lobby TV Behavior
`T < startTime` on today's date	UPCOMING	Warm Amber Pill	Shows meeting countdown and upcoming badge.
`startTime ≤ T < endTime`	ONGOING	Glowing Emerald Pill (Pulsing)	Neon green glowing border, active participant avatar.
`T ≥ endTime`	COMPLETED	Dual-tone Badge	Shows completed status & marks room as **AVAILABLE NOW**.
No active/upcoming meeting	AVAILABLE	Cyan/Teal Pill	Ready for immediate walk-in or new scheduling.

4. Security & Role-Based Access Control (RBAC)

Role Permissions Matrix

Feature / Route	Admin Role	Normal User Role
View Dashboard & Today's Meetings	Full Access	Full Access
View All Meetings & Filter Schedule	Full Access	Full Access
Create New Meeting (`POST /api/meetings`)	Allowed	Blocked (`403 Forbidden`)
Edit Meeting (`PUT /api/meetings/[id]`)	Allowed	Blocked (`403 Forbidden`)
Delete Meeting (`DELETE /api/meetings/[id]`)	Allowed	Blocked (`403 Forbidden`)
Manage Rooms (Add / Edit / Deactivate)	Allowed	Blocked (`403 Forbidden`)
Manage Users (`/users`)	Allowed (Full CRUD)	Blocked (`403 Forbidden`)
Edit Other Users' Names / Roles	Allowed	Blocked (`403 Forbidden`)
Edit Own Profile Name (`/profile`)	Allowed	Allowed (Self-Service)
View Live TV & Tablet Displays	Full Access	Full Access

Pre-Configured System Accounts

Name	User ID / Login	Password	Role	Notes
Pradhan	`Pradhan` or `pradhan@office.com`	`123`	ADMIN	Primary administrator account with full control.
Rahul Sharma	`admin@office.com` (USR001)	`Admin@123`	ADMIN	Secondary administrator account.
Priya Patel	`priya@office.com` (USR002)	`User@123`	USER	Normal employee user (read-only schedule access).

Authentication Implementation Mechanics

- **Session Storage:** Stored inside an HTTP-only, secure cookie named `mr_auth_token` with a 7-day expiration.
- **Token Re-signing on Name Change:** When a user updates their profile name in `/profile`, the backend immediately re-signs the JWT and updates the cookie so the user never has to log out to see their new name.
- **Password Security:** Passwords are never saved in plain text. Salted hashes are produced using `bcryptjs`.

5. Live Display System (TV & Tablet Screens)

1. All-Rooms TV Broadcast Display (`/display`)

Built specifically for 4K/1080p lobby TV screens and command center monitors.

- **Ambient Lighting & Glassmorphism:** Deep obsidian background (`#040714`) with multi-layered neon glow orbs (cyan, emerald, purple).
- **Signature Room Color Palettes:** Every room possesses its own distinct glowing neon jewel-tone card (Emerald, Amber, Cyan, Purple, Rose, Indigo) to make status recognition effortless from across a lobby.
- **Interactive Date Range Selector:** Clicking the header date badge opens an interactive popover to select `From Date` and `To Date` , or pick quick presets like "Today", "27 Aug", or "27–30 Aug Range".
- **Max 6 Items Initial Display Cap:** Prevents screen clutter by displaying a maximum of 6 cards initially.
- **"See More" Expand Button:** When more than 6 meetings exist in the date range, a glowing gradient button appears below the grid to view all scheduled sessions.
- **Live Ticking Clock:** Ticks every second with hydration mismatch guards.
- **15-Second Polling:** Automatically fetches the latest database state every 15 seconds without full page reload.

2. Room-Specific Outside-Door Tablet Screen (`/display/[roomId]`)

Built for 8-inch to 10-inch tablets mounted outside each conference room door (e.g. `/display/Room 101`).

- **Current Meeting Hero Card:** Large bold meeting title, time range, host avatar, and glowing status.
- **Today's Schedule Timeline:** Vertical timeline connecting upcoming meetings for that room with status indicators.
- **Noise Notice Footer:** "*Please keep noise to a minimum. Thank you!*".

3. Interactive Time Simulator

An embedded toolbar on the display pages that allows administrators to simulate different times of day without altering the real system clock:

- `09:30 AM` : Morning syncs upcoming.
- `01:30 PM` : Midday meetings.
- `02:35 PM` : Client Discussion ONGOING.
- `03:45 PM` : Team Meeting ONGOING.
- `04:30 PM` : Project Review ONGOING.
- `06:00 PM` : All meetings COMPLETED, rooms AVAILABLE.
- **"Use Real Clock" Button:** Instantly reverts back to the real system device time.

6. Complete REST API Specifications

Method & Endpoint	Auth	Description
`POST /api/auth/login`	Public	Logs in user via email, userId, or name with password. Sets JWT cookie.
`POST /api/auth/logout`	Public	Clears session cookie.
`GET /api/auth/me`	Session	Returns currently authenticated user profile.
`GET /api/meetings`	Public	Retrieves meetings with optional filters: `date`, `roomId`, `status`, `search`.
`POST /api/meetings`	Admin	Schedules meeting after running conflict validation. Returns `409` if room booked.
`GET /api/meetings/[id]`	Session	Returns single meeting details and assigned room information.
`PUT /api/meetings/[id]`	Admin	Updates meeting details after running conflict validation (excluding self).
`DELETE /api/meetings/[id]`	Admin	Deletes a meeting from the schedule.
`GET /api/rooms`	Session	Lists all rooms with meeting counts.
`POST /api/rooms`	Admin	Creates a new room with capacity, location, and room number.
`PUT /api/rooms/[id]`	Admin	Updates room information.
`DELETE /api/rooms/[id]`	Admin	Deletes room if no meetings exist; soft-deactivates (`inactive`) if meetings exist.
`GET /api/users`	Admin	Lists all employee and admin accounts.
`POST /api/users`	Admin	Registers a new user account with hashed password.
`PUT /api/users/[id]`	Self/Admin	Updates user name, email, or password. Admin can also change roles and status.
`DELETE /api/users/[id]`	Admin	Deletes user account (prevents self-deletion).
`GET /api/display`	Public	Returns room statuses and meetings between `startDate` and `endDate`.
`GET /api/display/[roomId]`	Public	Returns live schedule for an outside-the-door tablet.
`GET /api/settings`	Session	Returns office operating hours and display notices.
`POST /api/settings`	Admin	Updates office settings.

7. Codebase Directory Structure & Operations Guide

Directory Tree

```
meeting-room-system/
├── prisma/
│   ├── schema.prisma            # PostgreSQL schema (User, Room, Meeting, Setting)
│   └── seed.ts                  # Pre-seeded users (Pradhan, Rahul, Priya) and mockup meetings
├── src/
│   ├── app/
│   │   ├── layout.tsx           # Global RootLayout with brand favicon
│   │   ├── globals.css          # Tailwind CSS 4 styles
│   │   ├── page.tsx             # Root route redirect (/dashboard or /login)
│   │   ├── icon.svg             # Brand calendar/room SVG icon
│   │   ├── login/page.tsx       # Login page with 1-click credentials
│   │   ├── dashboard/page.tsx   # KPI dashboard & today's meetings
│   │   ├── meetings/
│   │   │   ├── page.tsx         # Filterable meetings list with delete modal
│   │   │   ├── create/page.tsx  # Create meeting with real-time conflict preview
│   │   │   └── [id]/            # Details & Edit meeting pages
│   │   ├── rooms/page.tsx       # Rooms list with capacity and add/edit modal
│   │   ├── users/page.tsx       # User management with admin name edit modal
│   │   ├── profile/page.tsx     # User profile with self-service name update form
│   │   ├── settings/page.tsx    # Office hours and TV banner configuration
│   │   ├── display/
│   │   │   ├── page.tsx         # TV All-Rooms Display with Date Range & "See More"
│   │   │   └── [roomId]/page.tsx # Room-specific tablet display
│   │   └── api/                 # All 11 REST route handlers
│   ├── components/
│   │   ├── layout/              # Sidebar.tsx, Header.tsx, AppLayout.tsx
│   │   └── ui/                  # StatusBadge.tsx, TimeSimulator.tsx
│   ├── lib/
│   │   ├── prisma.ts            # Prisma client singleton
│   │   ├── auth.ts              # JWT sign/verify, bcrypt hashing
│   │   └── meetingStatus.ts     # Conflict detection & status calculation algorithms
│   └── types/index.ts           # Shared TypeScript interfaces
├── Dockerfile                   # Multi-stage production Docker build
├── docker-compose.yml           # 1-command local & VPS PostgreSQL + App stack
├── DEPLOYMENT.md                # Complete Cloud Hosting Guide (Vercel, Neon, Railway, Render)
├── .env.example                 # Environment variable templates
├── .env                         # Local configuration (DATABASE_URL, AUTH_SECRET)
├── package.json                 # Dependencies, scripts ("postinstall": "prisma generate")
└── next.config.ts               # Next.js configuration (devIndicators: false)
```

How to Run Locally with PostgreSQL

1. **Extract the ZIP:** Unpack `meeting-room-system.zip` to your chosen directory.
2. **Set PostgreSQL Connection:** Ensure `DATABASE_URL` in `.env` points to your local or cloud PostgreSQL (e.g. Neon/Supabase).
3. **Install Dependencies:** Run `npm install`.
4. **Sync Database Schema:** Run `npx prisma db push`.

5. **Seed Initial Data:** Run `npm run db:seed`.
6. **Start Development Server:** Run `npm run dev`.
7. **Access Application:** Open `http://localhost:3000`.

> **Admin Access:** Login using Username: `Pradhan` and Password: `123`.

Production Cloud Hosting Options

- **Option 1: Vercel + Neon (Free Tier):** Create a free database on Neon.tech, set `DATABASE_URL` and `AUTH_SECRET` in Vercel project settings, and deploy with 1 click.
- **Option 2: Railway.app:** Add PostgreSQL database plugin, connect GitHub repository, set `DATABASE_URL` to `${{Postgres.DATABASE_URL}}`.
- **Option 3: Render.com:** Create managed PostgreSQL database and deploy Next.js Web Service.
- **Option 4: Docker / VPS:** Run `docker compose up -d` on any Linux/Windows server.